Assignment

Community Recreation Membership System

GenAI Level 1 — Limited AI Use ⚠️
This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the *Generative AI Use in Assessments* policy posted at the top of the Assignments section in Moodle.

Scenario

A community recreation centre needs a small **C# console application** to manage its members and their activities.

Each member has basic information, a membership type, a monthly fee, an optional locker number, and may participate in several activities.

Your application should demonstrate appropriate use of **basic types, strings, nullable types, regular enums, flag enums, and `StringBuilder`**.

Part 1 — Create the Member Model

Create a C# Console Application named:

`RecreationMembershipSystem`

Create a **`Member`** class containing appropriate properties for:

- Member ID
- Full Name
- Age
- Email
- Monthly Fee
- Membership Type
- Optional Locker Number
- Activities

Choose appropriate C# types for each property.

Create the following regular enum:

```
public enum MembershipType
{
    Standard,
    Student,
    Senior,
    Family
}
```

Use a **nullable value type** for the locker number so that a member can have either a locker number or no assigned locker.

Part 2 — Create an Activities Flags Enum

A member may participate in more than one activity.

Create:

```
[Flags]
0 references
public enum Activities
{
    0 references
    None = 0,
    0 references
    Swimming = 1,
    0 references
    Fitness = 2,
    0 references
    Yoga = 4,
    0 references
    Basketball = 8
}
```

[

Create at least **five different members** demonstrating different:

- membership types;
- monthly fees;
- locker assignments;
- activity combinations.

At least one member must have **no locker**, and several members must participate in multiple activities.

Use the bitwise OR operator (`|`) to combine activities.

For example:

```
Activities.Swimming | Activities.Yoga
```

Part 3 — Process Strings

Ask the user to enter a member's full name.

Process the input using appropriate string operations. Your application must demonstrate:

- **`Trim()`**
- **`ToUpper()`** or **`ToLower()`**
- **`Split()`**
- **`String.Join()`**
- string interpolation
- a string comparison

For example, the program should be able to process:

```
Alex Johnson
```

and correctly work with the cleaned name.

The string operations should have a meaningful purpose within the application rather than appearing as unrelated demonstrations.

Part 4 — Work with Nullable Types and Activities

Display each member's locker information.

If a locker exists:

```
Locker: 125
```

If no locker is assigned:

```
Locker: Not Assigned
```

Do not use **0** to represent an unassigned locker.

Also demonstrate flag operations by:

1. assigning multiple activities using `|`;

2. adding another activity using `|=`;
3. checking whether a member participates in a particular activity using `&`.

For example:

```
bool participatesInSwimming =
    (activities & Activities.Swimming) != 0;
```

Part 5 — Calculate Fees and Generate a Report

Calculate and display:

- each member's monthly fee;
- total monthly membership revenue;
- average monthly membership fee.

Use an appropriate numeric type for monetary calculations.

Then use **`StringBuilder`** to generate a formatted summary similar to:

```
COMMUNITY RECREATION CENTRE
MEMBERSHIP REPORT

Member: Alex Johnson
Membership: Student
Monthly Fee: $35.00
Activities: Swimming, Yoga
Locker: 125

Member: Maria Chen
Membership: Family
Monthly Fee: $75.00
Activities: Fitness, Basketball
Locker: Not Assigned
```

Convert the completed **`StringBuilder`** to a **`string`** before displaying the report.

Part 6 — Build and Test

Build and run the completed application.

Your tests must demonstrate:

- different membership types;
- one and multiple activities;
- members with and without lockers;
- string input containing extra whitespace;
- string comparison using different capitalization;
- correct total and average fee calculations.

Make sure the application compiles and produces clear, readable output.

Report and Screenshot Requirements

Submit a short report documenting your implementation. Include a **comprehensive set of screenshots** demonstrating the practical work, including:

- the **`Member`** class and selected data types;
- regular and **`[Flags]`** enums;
- nullable locker implementation;
- string processing;
- flag operations;
- **`StringBuilder`**;
- calculations;
- successful build and final application output.

Screenshots must demonstrate that the features were **implemented and tested**, not simply show source-code files.

Also briefly explain:

1. Why is a nullable type appropriate for **`LockerNumber`**?
2. Why is **`MembershipType`** a regular enum while **`Activities`** is a **`[Flags]`** enum?
3. Why do flag values use **`1, 2, 4, 8`**?
4. What is the difference between **`|`** and **`&`** for flags?
5. Why did you choose your numeric type for monetary values?

Submission and Presentation

Submit the complete assignment and report through **Moodle** and **respect the due date specified on Moodle**.

In addition to the Moodle submission, **the assignment must be presented to the professor to receive a grade**. You should be prepared to run the application, explain your code, and demonstrate the required C# features.

Assignments that are submitted but not presented will not be graded.

Include the required **GenAI Disclosure Statement** according to the Level 1 policy. Failure to include the required disclosure will result in a **grade of zero (0)**.

Rubric

Criterion	Weight
Member Model, Basic Types & Nullable Type	20%
String Processing & Manipulation	20%
Enums, Flags & Bitwise Operations	25%
Calculations, `StringBuilder` & Application Functionality	20%
Testing, Report, Screenshots & Presentation	15%
Total	**100%**